

How Many Bits Does a Smooth Quicksort Model Miss?

Distributional fit and the tree behind it

October 2, 2026

Abstract

Randomized Quicksort has an exact, discrete distribution of comparison counts at every finite input size. Our research program asks why a smooth curve can fit that distribution well while missing features of the process that generates it. We give two distinct answers in bits: a small, measurable mismatch between one-time count distributions, and a separate loss of next-step information when the growing tree is reduced to its total cost. The distinction identifies what a better continuous approximation must preserve.

From a good curve to an evolving model

Let C_n count comparisons when uniform-pivot Quicksort sorts n distinct keys. Its exact law obeys

$$C_n \stackrel{d}{=} n - 1 + C_{I_n} + C'_{n-1-I_n}, \quad I_n \sim \text{Uniform}\{0, \dots, n-1\}. \quad (1)$$

Our companion paper proves that an ordinary lognormal earns a higher expected log-density than an ordinary normal for every $n \geq 4$. Yet, after centering and scaling, C_n approaches a non-normal Quicksort law Q ; the fitted lognormal approaches a normal shape. A curve can therefore win at each finite size under one score and still miss the eventual shape.

We next modeled the *evolution* of the shape. Iterating the Quicksort limit's split-and-recombine rule from a point mass produces continuous laws $\nu_1, \nu_2, \dots$ that approach Q . Mixing adjacent generations gives a fractional interpolation. It fits central probabilities fairly well, but at small sizes understates skewness and the long right tail. Putting the finite- n split in (1) back into a one-step model improves both. This raises a precise question: how far is the smooth model from the exact count law?

A finite bit measure for the count distribution

Write $p_n(c) = \Pr(C_n = c)$ for the exact probability of integer count c . Let $F_{n,\theta}$ be the CDF of a continuous approximation expressed in *comparison-count units*. Ask the same question of both models by rounding the continuous output to the nearest integer:

$$q_{n,\theta}(c) = F_{n,\theta}(c + \tfrac{1}{2}) - F_{n,\theta}(c - \tfrac{1}{2}). \quad (2)$$

Now p_n and $q_{n,\theta}$ live on the same count scale. With $m = (p_n + q_{n,\theta})/2$, their Jensen–Shannon divergence in bits is

$$\text{JS}_2(p_n, q_{n,\theta}) = \tfrac{1}{2} D_{\text{KL},2}(p_n \| m) + \tfrac{1}{2} D_{\text{KL},2}(q_{n,\theta} \| m). \quad (3)$$

It is zero exactly when the rounded model gets every count probability right. Operationally, it is how much one observed count reveals about whether it came from Quicksort or the model, assuming either source was equally likely.

Input size n	Fractional interpolation	Finite-size split correction
16	0.00509	0.00186
32	0.00235	0.00135
64	0.00085	0.00070

Two independent Monte Carlo runs; Jensen–Shannon bits per rounded count.

Reading the table. At $n = 16$, restoring finite-size structure cuts this mismatch by about 63%. The $n = 16$ correction uses exact child laws through size 8; the $n = 32, 64$ corrections use moment-matched Q child laws. The numbers are exploratory Monte Carlo estimates, especially close at $n = 64$. Without the shared rounding rule, ordinary KL divergence is infinite simply because one law has atoms and the other is continuous; it is not a useful finite measure of fit.

A different loss: forgetting the growing tree

The same comparison count can arise from different binary search trees. Under the natural growing-tree coupling, let $K_n(d)$ be the number of empty insertion slots at depth d , and let D_{n+1} be the depth chosen next. Then

$$C_n = \sum_d dK_n(d) - 2n, \quad \Pr(D_{n+1} = d \mid K_n) = \frac{K_n(d)}{n+1}. \quad (4)$$

Knowing only C_n is like knowing the total cost while forgetting where the remaining slots are. Two trees with the same cost can give different chances for the next insertion depth. The exact expected predictive penalty, in bits, for retaining only cost is

$$L_n = I(D_{n+1}; K_n \mid C_n) = H(D_{n+1} \mid C_n) - H(D_{n+1} \mid K_n). \quad (5)$$

Equivalently, L_n is the smallest expected extra log-loss of any predictor that sees only C_n , relative to one that sees K_n . This is a property of the natural tree coupling, not of a particular interpolating curve.

Exact enumeration gives $L_{16} = 0.31545$ bits. The largest value through $n = 26$ occurs at $n = 14$, about 0.31866 bits, followed by a decline to $L_{26} = 0.27815$. Two independent large-tree simulations estimate roughly 0.06 bits at $n = 512$. The evidence points toward a shrinking one-step gap, but does not prove that it vanishes. Meanwhile, the number of hidden profile details can grow: $H(K_n \mid C_n)$ rises from 2.38 bits at $n = 14$ to 5.85 bits at $n = 26$. More of the tree is hidden by the total cost, yet less of that hidden detail helps predict the very next insertion.

What “close the gap” means

The two bit values answer different questions. Equation (3) tests *one-time distributional fit* after declaring the count resolution. Equation (5) tests *next-step prediction* after compressing the tree to one number. They cannot be added or compared as parts of a single information budget. In particular, matching every rounded count probability would close the first gap while leaving the second possible.

For the first goal, the next model should replace terminal- Q child laws by size-dependent child approximations, then score unseen input sizes against the exact recurrence. For the second, the open theoretical question is the large- n behavior of L_n . A continuous description can retain the tree profile, and a discrete count-only description can forget it. The informative distinction is therefore how much *state* the model keeps, alongside how closely it fits the count distribution.

Reproducibility. The companion paper source, `quicksort_*.py` experiment scripts, and dated `artifacts/` notes in this repository contain the recurrence calculations, simulations, and limitations behind these figures.